
TP 2.1 — GENERADORES DE NÚMEROS PSEUDOALEATORIOS

17 de junio de 2026

Tomás Andino

LEG: 52995

tomasandino12@gmail.com

Agustín Barroso Bollero

LEG: 52818

agusbarrosobollero@gmail.com

Facundo Gregoret

LEG: 52634

gregoretfacu18@gmail.com

Gerónimo Negri

LEG: 52991

geronimonegri@gmail.com

Bruno Schiffo

LEG: 53256

schiffobruno2233@gmail.com

Universidad Tecnológica Nacional — Facultad Regional Rosario

Zeballos 1341, S2000, Rosario, Argentina

Materia: Simulación — Año 2026

ABSTRACT

El presente trabajo introduce, explica y estudia la generación de números pseudoaleatorios en el contexto de la simulación computacional. Se implementaron en Python 3 distintos generadores: el Método de los Cuadrados Medios (*Middle Squares*), el Generador Congruencial Lineal (GCL) y se los comparó con el módulo `random` de la biblioteca estándar de Python. La calidad de cada generador fue evaluada mediante cuatro pruebas estadísticas: test de uniformidad (Chi-cuadrado), test de frecuencia, test de series y test de corridas. Los resultados obtenidos se analizaron y compararon tanto numéricamente como mediante visualizaciones gráficas.

Keywords Simulación · Números pseudoaleatorios · GCL · Mersenne Twister · Test Chi-cuadrado · Python

1. Introducción

1.1. ¿Qué es un número pseudoaleatorio?

Un número pseudoaleatorio es un número generado por un algoritmo determinista que imita el comportamiento de un número verdaderamente aleatorio. Se dice “pseudo” porque no hay aleatoriedad real: dado el mismo punto de partida (semilla), el algoritmo siempre produce la misma secuencia. Sin embargo, si los parámetros son buenos, la secuencia es estadísticamente indistinguible de una aleatoria verdadera.

La diferencia con un número realmente aleatorio es que este último proviene de un fenómeno físico impredecible o muy difícil de reproducir (ruido térmico, decaimiento radiactivo, etc.), mientras que el pseudoaleatorio es completamente reproducible.

1.2. Importancia en simulación

En simulación, casi todo depende de números aleatorios: modelar llegadas de clientes, tiempos de servicio, fallas de equipos, variaciones de demanda, etc. Usar números verdaderamente aleatorios es impráctico porque no son reproducibles ni controlables.

Los números pseudoaleatorios resuelven esto porque permiten repetir exactamente una simulación cambiando solo la semilla, comparar escenarios bajo las mismas condiciones, depurar errores reproduciendo la misma corrida, y generar grandes volúmenes de números de forma rápida y económica computacionalmente.

1.3. Objetivos del trabajo

- Estudiar e implementar distintos métodos de generación de números pseudoaleatorios: Método de los Cuadrados Medios y Generador Congruencial Lineal (GCL).
- Comparar su comportamiento estadístico mediante tests de calidad: uniformidad, frecuencia, series y corridas.
- Contrastar los generadores implementados contra el módulo `random` de Python como referencia.
- Analizar las ventajas y limitaciones de cada método en el contexto de simulación.

2. Marco Teórico

2.1. Generadores de números aleatorios reales

Los sistemas informáticos son deterministas y están diseñados para dar resultados predecibles a partir de una entrada. Por este motivo, generar aleatoriedad pura utilizando únicamente software es imposible. Para lograrlo, los verdaderos generadores de números aleatorios extraen “entropía” midiendo fenómenos físicos del mundo real.

Conseguir esta aleatoriedad es vital para la ciberseguridad. Cada clave de encriptación debe ser totalmente impredecible para evitar que un atacante la descubra. Un ejemplo clásico es `RANDOM.ORG`, que genera números con el ruido atmosférico (2).

Sin embargo, a nuestro grupo de trabajo le interesó una solución mucho más curiosa, el sistema de Cloudflare (8). Esta empresa resolvió el problema de la entropía utilizando lámparas de lava. La cera en estas lámparas se mueve de forma aleatoria y nunca repite exactamente la misma forma. Cloudflare instaló aproximadamente 100 lámparas en una pared de su sede central y las graba constantemente con una cámara de video.

Como las computadoras almacenan cada fotograma como una matriz de números según el valor y color de cada píxel, los cambios en el movimiento de la lava se convierten al instante en una cadena numérica totalmente aleatoria.



Figura 1: Pared de lámparas de lava en la sede de Cloudflare, utilizada como fuente de entropía para encriptación.

2.2. Generadores de números pseudoaleatorios

2.2.1. Método de los Cuadrados Medios (*Middle Squares*)

En 1946, John von Neumann propuso aprovechar las operaciones matemáticas de las computadoras para crear secuencias de números pseudoaleatorios. Así nació el método de los Cuadrados Medios (1).

El algoritmo clásico para generar secuencias de 4 dígitos es muy directo:

- 1- Se elige un número inicial de 4 dígitos conocido como semilla.
- 2- Se eleva esta semilla al cuadrado para obtener un resultado de 8 dígitos. Si el número resultante tiene menos cifras, se le agregan ceros a la izquierda hasta completar los 8 espacios.
- 3- Se extraen los 4 dígitos centrales. Este nuevo valor es el número pseudoaleatorio generado y pasa a ser la semilla del siguiente ciclo.

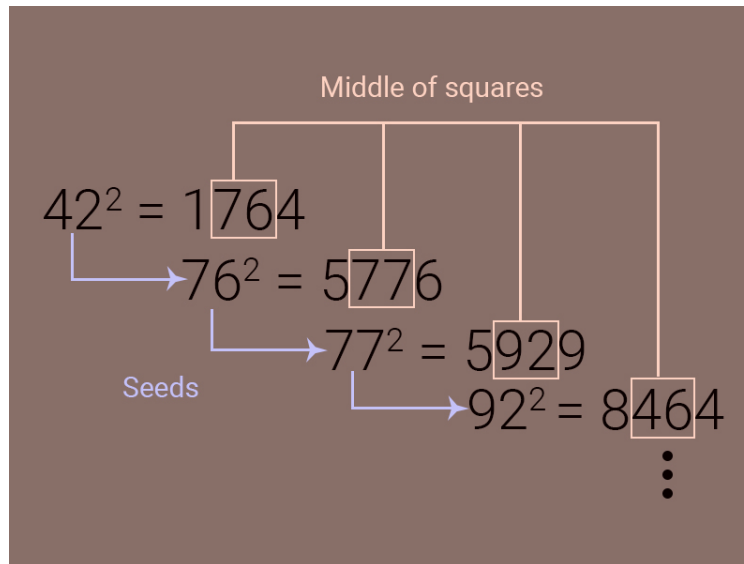


Figura 2: Esquema del algoritmo del Método de los Cuadrados Medios.

2.2.2. Generador Congruencial Lineal (GCL)

Introducidos por D.H. Lehmer en 1949, los Generadores Congruenciales Lineales son algoritmos muy populares la simulación (1). La calidad de la secuencia numérica depende de una buena elección de sus parámetros iniciales.

El algoritmo genera cada nuevo número basándose en el anterior mediante la siguiente ecuación:

$$X_{n+1} = (aX_n + c) \pmod{m} \quad (1)$$

Donde sus componentes matemáticos son:

- **Módulo (m):** Debe ser mayor a cero. Se busca que sea un número sumamente grande porque define el límite máximo del ciclo o periodo del generador. Para maximizar la velocidad de procesamiento se suelen utilizar potencias de 2 (como 2^{32} o 2^{64}) ya que coinciden con el tamaño de palabra de la computadora y permiten resolver la operación módulo simplemente truncando bits.
- **Multiplicador (a):** Debe ser un valor mayor o igual a cero y menor que el módulo.
- **Incremento (c):** Debe ser menor o igual al módulo.
- **Semilla (X_0):** Representa el valor de inicio de la secuencia y debe ser mayor o igual a cero pero menor que el módulo.

A pesar de su gran velocidad computacional los GCL tienden a exhibir defectos ya documentados. Siempre generan hiperplanos paralelos según la dimensión de análisis. Explicándolo en dos dimensiones, si tomamos la secuencia de valores producidos por el GCL y los agrupamos de a pares para formar coordenadas cartesianas (X_n, X_{n+1}) no logramos una dispersión caótica que cubra todo el plano de manera uniforme. En su lugar los puntos inevitablemente se agrupan formando un patrón de líneas rectas diagonales y paralelas. Los amplios espacios vacíos que quedan entre estas franjas representan zonas donde el algoritmo es matemáticamente incapaz de generar resultados.

2.2.3. Módulo random de Python - Mersenne Twister

Desarrollado en 1997 por Makoto Matsumoto y Takuji Nishimura, el Mersenne Twister es el algoritmo base que impulsa el módulo random de Python (9). Su nombre surge de los números primos de Mersenne, los cuales se definen con la forma $2^p - 1$ donde p también es un número primo. Este generador es uno de los más evaluados del mundo. En su implementación para Python produce números de punto flotante con 53 bits de precisión y destaca por tener un periodo de $2^{19937} - 1$. Esta longitud garantiza que el algoritmo puede generar una cantidad masiva de valores antes de repetir la secuencia lo que lo hace ideal para simulaciones complejas.

Funcionamiento del algoritmo

El método opera manteniendo un estado interno compuesto por un arreglo de 624 números enteros de 32 bits. La generación de los números aleatorios se logra mediante una técnica de manipulación de bits estructurada en tres fases:

- **Inicialización:** Se introduce un valor semilla que configura el estado interno inicial del sistema.
- **Torsión (*Twisting*):** El estado se actualiza aplicando una transformación matemática específica sobre los bits. Este paso es el que asegura que la nueva secuencia generada tenga un comportamiento estadísticamente aleatorio.
- **Salida (*Tempering*):** Para entregar el resultado definitivo se aplica una función de templado sobre el estado. Este proceso desordena y mezcla los bits una última vez para producir el número final.

A pesar de sus aplicaciones de simulación, el Mersenne Twister sigue siendo un algoritmo completamente determinista. Debido a esto la documentación oficial de Python brinda una warning “los generadores pseudoaleatorios de este módulo jamás deben usarse para propósitos de seguridad o criptografía” (4). Para generar claves seguras o manejar datos sensibles la propia documentación recomienda utilizar el módulo `secrets` diseñado específicamente para esos fines.

La formula matemática de este generador se omite por estar basada en manipulaciones complejas de bits que escapan a los objetivos de este trabajo.

2.3. Tests de calidad para generadores

Para comprobar esta pseudoaleatoriedad usamos gráficos y pruebas estadísticas. Estas herramientas nos marcan qué tan cerca de ser realmente aleatorios son los números generados.

2.3.1. Test de Uniformidad (Chi-cuadrado)

El objetivo de esta prueba es responder a una pregunta central: ¿los números generados se ajustan a la distribución que esperábamos o son puro ruido?

Para averiguarlo el procedimiento es directo:

- **Crear intervalos:** Tomamos todo el rango de números posibles y lo dividimos en subgrupos o cajas del mismo tamaño.
- **Frecuencia Observada (O_i):** Contamos la cantidad exacta de números que cayeron en cada caja.
- **Frecuencia Esperada (E_i):** Calculamos cuántos números debían caer en cada caja si el algoritmo funcionara en un escenario ideal.
- **Cálculo del desvío:** Medimos la diferencia entre lo observado y lo esperado para evaluar qué tan lejos quedó la realidad de la teoría.

En lugar de resolver ecuaciones a mano evaluamos el error total usando la distribución Chi-cuadrado. El programa de análisis calcula nuestro desvío y se fija en qué parte de la curva cae. Si el valor queda en la zona de aceptación significa que los desvíos son normales y el generador aprueba. Si cae muy lejos el error es demasiado grande y el algoritmo falla.

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i} \quad (2)$$

2.3.2. Test de Frecuencia

Es el paso previo y el complemento del Chi-cuadrado. Su meta es detectar desvíos puntuales en la cantidad de apariciones.

El histograma es la forma gráfica de ver este test y la base de la uniformidad. Te muestra de un vistazo cuántos números cayeron en cada intervalo y lo compara con una línea de referencia que marca lo que se esperaba de forma teórica.

2.3.3. Test de Series

Un generador puede pasar la uniformidad pero fallar si sus números dependen entre sí. Este test evalúa la independencia secuencial analizando el espacio en varias dimensiones.

Para implementarlo agrupamos los números generados de a pares simulando coordenadas (x, y) . Después dividimos el plano en una grilla. Contamos cuántos pares caen en cada celda y aplicamos la misma lógica del Chi-cuadrado para ver si llenan el espacio de manera pareja o si dejan huecos.

El gráfico de dispersión o *scatter plot* es la visualización exacta de este test. Ubica los puntos en el plano para mostrar si se distribuyen bien o si forman patrones geométricos.

2.3.4. Test de Corridas

Este test evalúa cómo oscila la secuencia. Un buen algoritmo tiene que alternar entre subir y bajar de forma impredecible sin quedarse trabado en rachas de puro crecimiento o decrecimiento.

Para la prueba comparamos cada número con el anterior. Si es mayor anotamos un signo positivo y si es menor uno negativo. Una “corrida” es una racha ininterrumpida de signos iguales. Luego contamos el total de corridas obtenidas y lo comparamos contra una media esperada. Si el cálculo final arroja un valor Z dentro del margen de aceptación confirmamos que la secuencia oscila correctamente.

$$\mu_R = \frac{2n - 1}{3}, \quad \sigma_R^2 = \frac{16n - 29}{90} \quad (3)$$

3. Implementación

Para llevar a cabo la investigación se programaron los algoritmos en Python. Todos los generadores fueron diseñados para devolver una lista de números en el intervalo cerrado-abierto $[0, 1)$.

3.1. Método de los Cuadrados Medios

```

1 def cuadrados_medios(x0, n):
2     D = len(str(x0))
3     secuencia = []
4     x = x0
5     for _ in range(n):
6         y = x ** 2
7         x = (y // 10**(D//2)) % (10**D)
8         if x == 0:
9             print("Generador degenerado: X=0")
10            break
11        secuencia.append(x / 10**D)
12    return secuencia

```

Listing 1: Implementación del Método de los Cuadrados Medios en Python

El algoritmo arranca calculando la cantidad de dígitos de la semilla inicial. Adentro del ciclo eleva el número al cuadrado y usa la división entera combinada con la operación módulo para recortar los números del centro. Incluye una validación para detener el proceso si el valor cae en cero evitando que el generador colapse. Finalmente divide el resultado por la potencia de diez correspondiente para devolver flotantes.

3.2. Generador Congruencial Lineal (GCL)

```

1 def gcl(m, a, c, x0, n):
2
3     # Validaciones omitidas...
4     # Hull-Dobell 1: c y m deben ser coprimos
5     if math.gcd(c, m) != 1:

```

```

6         print(f"Hull-Dobell 1 falla: mcd(c={c}, m={m}) = {math.gcd(c,m)} != 1");
7         return None
8
9     # Hull-Dobell 2: cada primo que divide a m debe dividir a (a-1)
10    n_temp = m
11    d = 2
12    while d * d <= n_temp:
13        if n_temp % d == 0:
14            if (a - 1) % d != 0:
15                print(f"Hull-Dobell 2 falla: (a-1)={a-1} no es divisible por {d}");
16                return None
17            while n_temp % d == 0:
18                n_temp //= d
19            d += 1
20    if n_temp > 1 and (a - 1) % n_temp != 0:
21        print(f"Hull-Dobell 2 falla: (a-1)={a-1} no es divisible por {n_temp}");
22        return None
23
24    # Hull-Dobell 3: si 4 | m, entonces 4 | (a-1)
25    if m % 4 == 0 and (a - 1) % 4 != 0:
26        print(f"Hull-Dobell 3 falla: 4 | m={m} pero 4 no divide a (a-1)={a-1}");
27        return None
28
29    # Generacion
30    secuencia = []
31    x = x0
32    for _ in range(n):
33        x = (a * x + c) % m
34        secuencia.append(x / m)
35    return secuencia

```

Listing 2: Implementación del Generador Congruencial Lineal en Python

La función original se divide en dos grandes bloques. Primero valida los parámetros ingresados mediante el Teorema de Hull-Dobell comprobando que el módulo, el multiplicador y el incremento sean coprimos para asegurar el periodo máximo. Una vez superadas las validaciones entra generación donde aplica la ecuación lineal y divide el resultado por el módulo para normalizar los valores.

3.3. random de Python (Mersenne Twister)

```

1 def python_random(n):
2     import random
3     return [random.random() for _ in range(n)]

```

Listing 3: Generación nativa utilizando el módulo random

Al ser el algoritmo nativo del lenguaje la implementación es importar la librería estándar. Se ejecuta una función interna de Python (escrita en c según la documentación) la cual ejecuta el Mersenne Twister en segundo plano y devuelve los valores listos para usar.

3.4. Implementación de los tests de calidad

Para evaluar el comportamiento de los generadores dividimos la lógica en cuatro funciones independientes. Todas reciben la lista de números generados en el rango $[0, 1)$ y devuelven los estadísticos correspondientes.

3.4.1. Test de Uniformidad (Chi-cuadrado)

```

1 def test_uniformidad(secuencia, intervalos=10):
2     n = len(secuencia)
3     esperado = n / intervalos
4     observado = [0] * intervalos

```

```

6     for u in secuencia:
7         indice = min(int(u * intervalos), intervalos - 1)
8         observado[indice] += 1
9
10    chi2 = sum((o - esperado) ** 2 / esperado for o in observado)
11    return chi2

```

Listing 4: Implementación del Test de Uniformidad

Este test clasifica los valores multiplicando cada numero por la cantidad de intervalos deseados. Luego aplica la sumatoria de las diferencias al cuadrado entre lo observado y lo esperado para obtener el error total.

3.4.2. Test de Frecuencia

```

1  def test_frecuencia(secuencia, intervalos=10):
2      n = len(secuencia)
3      esperado = n / intervalos
4      conteo = [0] * intervalos
5
6      for u in secuencia:
7          indice = min(int(u * intervalos), intervalos - 1)
8          conteo[indice] += 1
9
10     desviaciones = [abs(c - esperado) for c in conteo]
11     return max(desviaciones), sum(desviaciones) / intervalos

```

Listing 5: Implementación del Test de Frecuencia

Reutiliza la lógica del test anterior pero su objetivo final se enfoca exclusivamente en calcular el desvío máximo y el desvío promedio absoluto de las ocurrencias.

3.4.3. Test de Series

```

1  def test_series(secuencia, intervalos=5):
2      n = len(secuencia)
3      esperado = (n - 1) / (intervalos ** 2)
4      conteo = [[0] * intervalos for _ in range(intervalos)]
5
6      for i in range(n - 1):
7          fila = min(int(secuencia[i] * intervalos), intervalos - 1)
8          col = min(int(secuencia[i + 1] * intervalos), intervalos - 1)
9          conteo[fila][col] += 1
10
11     chi2 = sum(
12         (conteo[f][c] - esperado) ** 2 / esperado
13         for f in range(intervalos)
14         for c in range(intervalos)
15     )
16     return chi2

```

Listing 6: Implementación del Test de Series

Analiza la secuencia bidimensionalmente. Agrupa los números de a pares (X, Y) y las ubica en una gráfica para evaluar si llenan el espacio de forma pareja.

3.4.4. Test de Corridas

```

1  def test_corridas(secuencia):
2      n = len(secuencia)
3      corridas = 1
4
5      for i in range(1, n):
6          if secuencia[i] != secuencia[i - 1]:

```

```

7         if (secuencia[i] > secuencia[i - 1]) != (secuencia[i - 1] > secuencia[
            i - 2] if i > 1 else True):
8             corridas += 1
9
10        media_esperada = (2 * n - 1) / 3
11        varianza_esperada = (16 * n - 29) / 90
12        z = (corridas - media_esperada) / math.sqrt(varianza_esperada)
13
14        return z

```

Listing 7: Implementación del Test de Corridas

Recorre la lista completa comparando cada número con su anterior. A través de operadores lógicos detecta los cambios de tendencia contando las rachas de crecimiento o decrecimiento para finalmente calcular el Z .

4. Resultados y Análisis

4.1. Aplicación de los tests al GCL

Se evaluó el Generador Congruencial Lineal con los siguientes parámetros:

Parámetro	Valor
Módulo (m)	$2^{32} = 4,294,967,296$
Multiplicador (a)	1,664,525
Incremento (c)	1,013,904,223
Semilla (X_0)	42
n	1000

Cuadro 1: Parámetros del GCL utilizados.

Estos parámetros satisfacen las tres condiciones del Teorema de Hull-Dobell, garantizando un período completo de 2^{32} valores antes de repetirse.

Test de Uniformidad

Se obtuvo un estadístico $\chi^2 = 12,00$ con 9 grados de libertad. Dado que este valor es menor al valor crítico de 16,92 para un nivel de significancia del 95 %, no se rechaza la hipótesis de uniformidad. Las frecuencias observadas por intervalo fueron:

Intervalo	Observado	Esperado
[0.0, 0.1)	77	100
[0.1, 0.2)	102	100
[0.2, 0.3)	109	100
[0.3, 0.4)	97	100
[0.4, 0.5)	102	100
[0.5, 0.6)	116	100
[0.6, 0.7)	84	100
[0.7, 0.8)	106	100
[0.8, 0.9)	103	100
[0.9, 1.0)	104	100

Cuadro 2: Frecuencias observadas vs esperadas — GCL.

El intervalo [0,0; 0,1) fue el más alejado del valor esperado con 77 ocurrencias, aunque no suficiente para rechazar la uniformidad.

Test de Frecuencia

El desvío máximo obtenido fue de 23,00 en el intervalo [0,0; 0,1), con un desvío promedio de 8,40. Estos valores representan un desvío máximo del 23 % respecto al valor esperado, considerado aceptable para $n = 1000$.

Test de Series

Se obtuvo un estadístico $\chi^2 = 19,94$ con 24 grados de libertad, inferior al valor crítico de 36,42. Esto indica que los pares consecutivos (U_i, U_{i+1}) se distribuyen de forma uniforme en la grilla 5×5 , sin evidencia de dependencia entre valores sucesivos.

Test de Corridas

Se contabilizaron 683 corridas sobre las 666,33 esperadas, resultando en un estadístico $Z = 1,25$. Como $|Z| = 1,25 < 1,96$, no se rechaza la aleatoriedad al 95 % de confianza. La secuencia alterna correctamente entre valores crecientes y decrecientes.

4.2. Aplicación de los tests a Cuadrados Medios

Se evaluó el Método de los Cuadrados Medios con la siguiente semilla:

Parámetro	Valor
Semilla (X_0)	1006
n	1000

Cuadro 3: Parámetros de Cuadrados Medios utilizados.

Test de Uniformidad

Se obtuvo un estadístico $\chi^2 = 1382,84$ con 9 grados de libertad, valor que supera ampliamente el valor crítico de 16,92. Se rechaza la hipótesis de uniformidad. Las frecuencias observadas revelan una concentración marcada en ciertos intervalos:

Intervalo	Observado	Esperado
[0,0, 0,1)	7	100
[0,1, 0,2)	5	100
[0,2, 0,3)	246	100
[0,3, 0,4)	4	100
[0,4, 0,5)	242	100
[0,5, 0,6)	0	100
[0,6, 0,7)	242	100
[0,7, 0,8)	5	100
[0,8, 0,9)	246	100
[0,9, 1,0)	3	100

Cuadro 4: Frecuencias observadas vs esperadas — Cuadrados Medios.

Los valores se concentran casi exclusivamente en los intervalos [0,2; 0,3), [0,4; 0,5), [0,6; 0,7) y [0,8; 0,9), mientras que el resto permanece prácticamente vacío.

Test de Frecuencia

El desvío máximo fue de 146,00, con un desvío promedio de 115,20. Ambos valores son extremadamente elevados respecto al valor esperado de 100 por intervalo, confirmando la no uniformidad observada en el test anterior.

Test de Series

Se obtuvo un estadístico $\chi^2 = 4829,80$ con 24 grados de libertad, muy superior al valor crítico de 36,42. Los pares consecutivos (U_i, U_{i+1}) no se distribuyen uniformemente en la grilla 5×5 , evidenciando una fuerte dependencia entre valores sucesivos.

Test de Corridas

Se contabilizaron 496 corridas sobre las 666,33 esperadas, resultando en un estadístico $Z = -12,79$. Como $|Z| \gg 1,96$, se rechaza la aleatoriedad al 95 % de confianza. El valor negativo indica que la secuencia tiene muy pocas corridas, es decir, permanece en las mismas regiones sin oscilar correctamente.

4.3. Aplicación de los tests al Mersenne Twister

El módulo random de Python implementa internamente el algoritmo Mersenne Twister. Al tratarse de la implementación nativa del lenguaje, no requiere parámetros definidos por el usuario. La semilla es asignada automáticamente por el sistema en cada ejecución.

Parámetro	Valor
Semilla (X_0)	Asignada automáticamente por el sistema
n	1000

Cuadro 5: Parámetros del Mersenne Twister utilizados.

Test de Uniformidad

Se obtuvo un estadístico $\chi^2 = 6,60$ con 9 grados de libertad, valor considerablemente inferior al valor crítico de 16,92. No se rechaza la hipótesis de uniformidad. Las frecuencias observadas fueron:

Intervalo	Observado	Esperado
[0.0, 0.1)	118	100
[0.1, 0.2)	102	100
[0.2, 0.3)	97	100
[0.3, 0.4)	105	100
[0.4, 0.5)	103	100
[0.5, 0.6)	99	100
[0.6, 0.7)	100	100
[0.7, 0.8)	96	100
[0.8, 0.9)	96	100
[0.9, 1.0)	84	100

Cuadro 6: Frecuencias observadas vs esperadas — Mersenne Twister.

Test de Frecuencia

El desvío máximo fue de 18,00 en el intervalo [0,0; 0,1), con un desvío promedio de 5,60.

Test de Series

Se obtuvo un estadístico $\chi^2 = 13,24$ con 24 grados de libertad, muy inferior al valor crítico de 36,42. Los pares consecutivos se distribuyen uniformemente en la grilla 5×5 , sin evidencia de dependencia entre valores sucesivos.

Test de Corridas

Se contabilizaron 667 corridas sobre las 666,33 esperadas, resultando en un estadístico $Z = 0,05$, indicando que la secuencia oscila con una frecuencia prácticamente idéntica a la esperada teóricamente.

4.4. Tabla comparativa de generadores

El siguiente cuadro resume los estadísticos obtenidos al aplicar los cuatro tests a cada generador con $n = 1000$.

Generador	Uniformidad χ^2	Frec. desvío máx.	Series χ^2	Corridas Z
Middle Squares	1382,84	146,00	4829,80	-12,79
GCL	12,00	23,00	19,94	1,25
Mersenne Twister	6,60	18,00	13,24	0,05
Valor crítico (95 %)	16,92	—	36,42	$ Z < 1,96$

Cuadro 7: Comparación de estadísticos de los tests para cada generador ($n = 1000$).

4.5. Visualizaciones gráficas

Para las visualizaciones se utilizaron dos valores de n distintos según el tipo de gráfico. Los histogramas se generaron con $n = 1000$, el mismo valor utilizado en los tests estadísticos, de forma que las frecuencias observadas sean directamente comparables con los resultados anteriores. Los scatter de pares consecutivos se generaron con $n = 5000$, ya que esta visualización requiere mayor densidad de puntos para revelar correctamente la estructura de distribución en el plano: con pocos puntos el espacio queda muy vacío y no se pueden apreciar patrones ni confirmar la ausencia de ellos.

4.5.1. GCL

El histograma confirma visualmente lo observado en los tests: las barras se mantienen cercanas al valor esperado de 100, sin concentraciones notables en ningún intervalo particular.

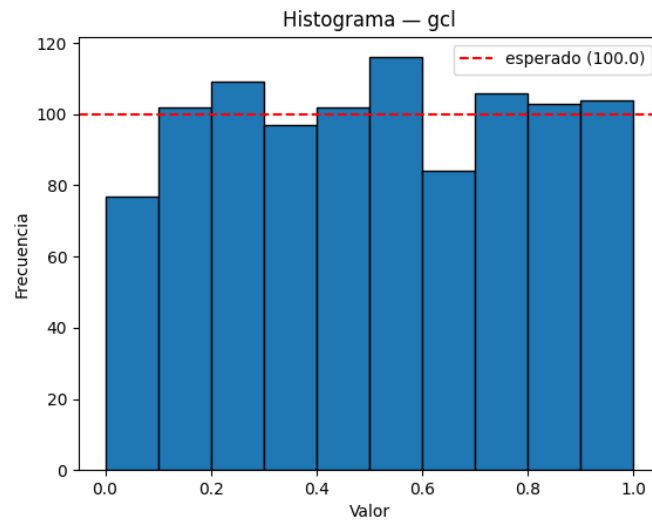


Figura 3: Histograma — GCL ($n = 1000$).

El scatter de pares consecutivos muestra una distribución sin patrones geométricos evidentes, coherente con los buenos resultados obtenidos en el test de series.

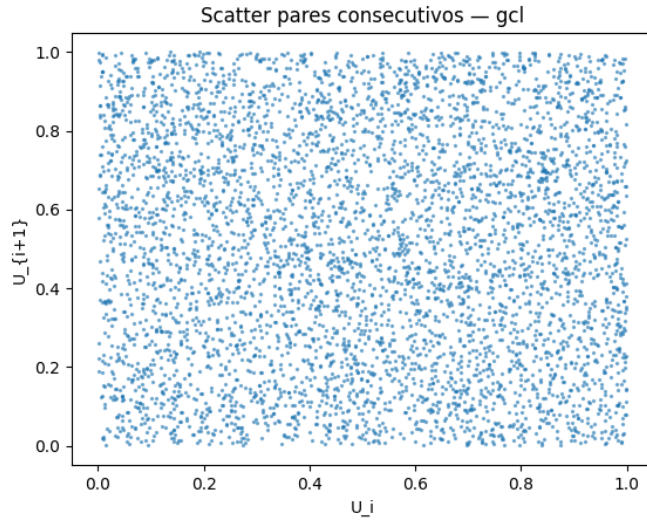


Figura 4: Scatter pares consecutivos — GCL ($n = 5000$).

Para ilustrar el problema de los hiperplanos mencionado en el marco teórico, se ejecutó el GCL con parámetros inadecuados ($m = 2^{32}$, $a = 5$, $c = 1$, $x_0 = 42$). El scatter resultante evidencia claramente la estructura geométrica característica del algoritmo:

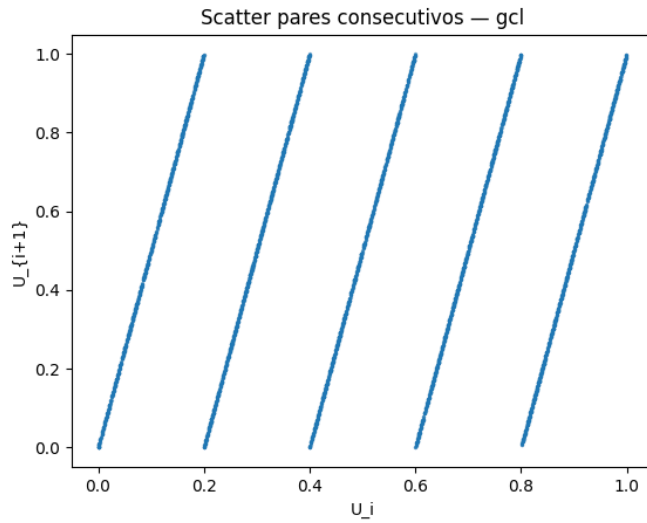


Figura 5: Scatter pares consecutivos — GCL con parámetros inadecuados. Se observan las líneas diagonales paralelas características del algoritmo.

4.5.2. Cuadrados Medios

El histograma refleja de forma contundente el fracaso del generador: cuatro intervalos concentran casi la totalidad de los valores mientras el resto permanece prácticamente vacío, en particular el intervalo $[0,5; 0,6)$ que no registró ninguna ocurrencia.

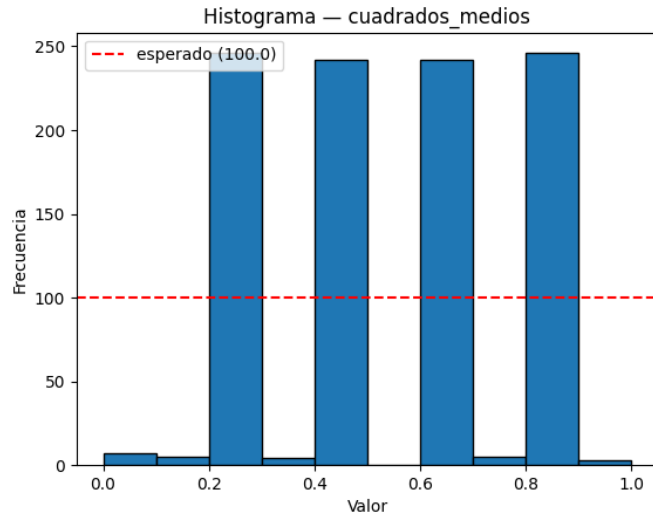


Figura 6: Histograma — Cuadrados Medios ($n = 1000$).

El scatter evidencia la fuerte dependencia entre valores consecutivos: los puntos no cubren el plano de forma uniforme sino que se agrupan en regiones específicas, confirmando el alto valor de χ^2 obtenido en el test de series. Cabe destacar que, a pesar de haberse solicitado $n = 5000$ puntos, el gráfico muestra una cantidad notablemente menor debido a que el generador entró en un ciclo corto, repitiendo indefinidamente los mismos valores. Los mismos pares (U_i, U_{i+1}) se superponen en las mismas posiciones del plano.

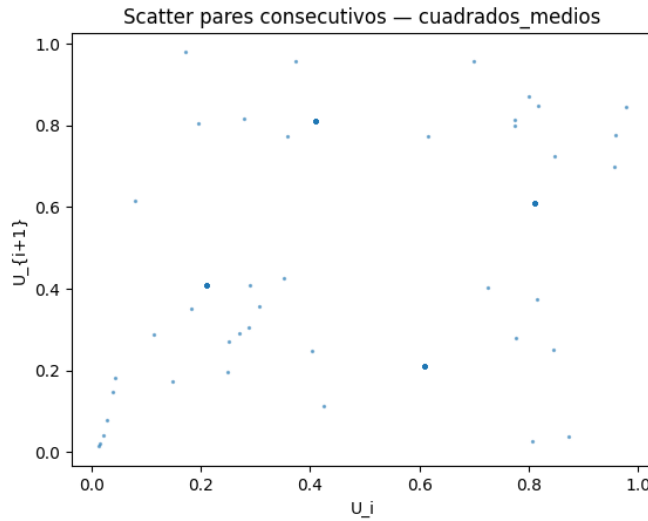


Figura 7: Scatter pares consecutivos — Cuadrados Medios ($n = 5000$).

Para ilustrar la sensibilidad del método a la semilla, se ejecutó el generador con $X_0 = 1234$, caso en el que el algoritmo degeneró completamente al llegar a cero en pocas iteraciones, interrumpiendo la generación antes de producir los n valores solicitados. A diferencia de $X_0 = 1006$ donde el generador ciclaba, aquí directamente colapsa. El scatter resultante muestra apenas unos pocos puntos, correspondientes a los valores generados antes de la degeneración.

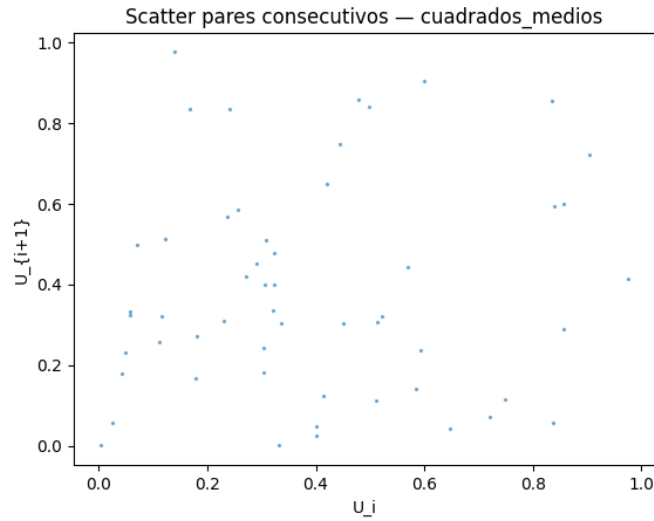


Figura 8: Scatter pares consecutivos — Cuadrados Medios con semilla degenerada ($X_0 = 1234$). El generador colapsó a cero en pocas iteraciones.

4.5.3. Mersenne Twister

El histograma es consistente con el estadístico $\chi^2 = 6,60$ obtenido en el test de uniformidad, el menor de los tres generadores. Si bien visualmente algunas barras muestran desvíos apreciables, el test estadístico confirma que estas variaciones se encuentran dentro de los márgenes esperados para $n = 1000$.

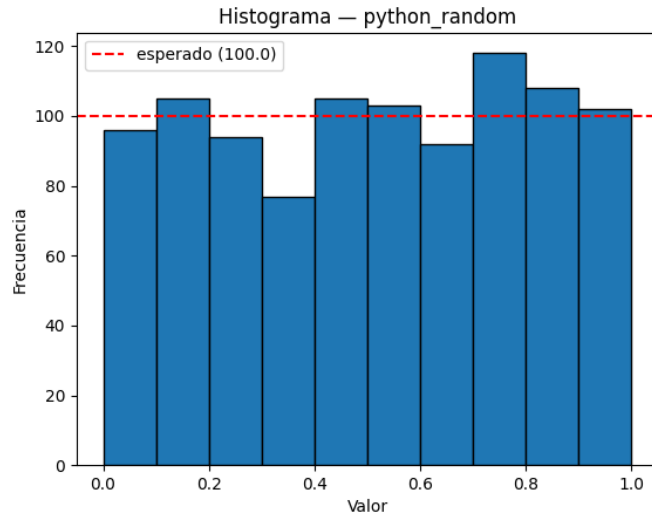


Figura 9: Histograma — Mersenne Twister ($n = 1000$).

El scatter presenta una distribución uniforme y sin estructuras geométricas visibles, contrastando claramente con los patrones observados con los hiperplanos del GCL y con la concentración en regiones del Método de Cuadrados Medios. Esto confirma la ausencia de dependencia entre valores consecutivos.

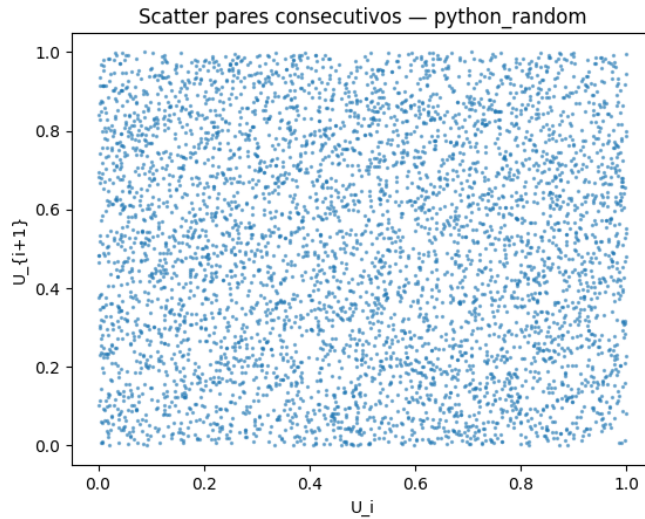


Figura 10: Scatter pares consecutivos — Mersenne Twister ($n = 5000$).

5. Conclusiones

El presente trabajo permitió estudiar, implementar y comparar tres generadores de números pseudoaleatorios mediante cuatro tests estadísticos, cumpliendo los objetivos planteados en la introducción.

En cuanto a los resultados, el Mersenne Twister obtuvo los mejores valores en todos los tests. Esto lo confirma como el generador de mayor calidad estadística de los tres. Garantiza resultados estadísticamente correctos independientemente de la semilla utilizada. Esto se debe a que su diseño interno fue optimizado matemáticamente para producir secuencias de alta calidad de forma inherente, sin requerir configuración por parte del usuario. Estos resultados lo confirman como el más adecuado para simulación.

El GCL aprobó los cuatro tests y sus resultados fueron comparables al Mersenne Twister, con valores dentro de los márgenes de aceptación en todas las pruebas. Sin embargo, no son equivalentes: la comparación con los parámetros inapropiados ($a = 5$, $c = 1$) dejó en evidencia que la calidad del GCL depende completamente de la elección de sus parámetros. Una mala elección no solo reduce el período sino que produce secuencias con estructura geométrica visible, como demostraron los hiperplanos observados en el scatter. Esta sensibilidad a los parámetros es la principal diferencia respecto al Mersenne Twister, cuya calidad no depende de configuración alguna por parte del usuario.

El Método de los Cuadrados Medios mostró exactamente las deficiencias esperadas teóricamente: falló los cuatro tests de forma contundente. Además, el análisis con distintas semillas reveló dos modos de fallo diferenciados: con $X_0 = 1006$ el generador entra en un ciclo corto que concentra los valores en regiones específicas, mientras que con $X_0 = 1234$ colapsa directamente a cero en pocas iteraciones. Esto demuestra que el método no solo produce secuencias de mala calidad estadística, sino que su comportamiento es impredecible y depende críticamente de la semilla elegida, sin ninguna garantía de funcionamiento aceptable para simulación.

Respecto a las limitaciones del trabajo, se utilizó $n = 1000$ y se aplicaron únicamente cuatro tests, lo cual constituye una evaluación parcial. A pesar de esto, las diferencias entre generadores fueron lo suficientemente claras como para validar los objetivos planteados y arribar a conclusiones sólidas sobre la calidad y aplicabilidad de cada algoritmo en el contexto de la simulación computacional.

Referencias

- [1] Riva Quiroga, T. *Estadística Computacional — Números Pseudoaleatorios*. Disponible en: <https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html>
- [2] Haahr, M. *Statistical Analysis of RANDOM.ORG*, 2005. Disponible en: <https://www.random.org/analysis/Analysis2005.pdf>
- [3] RANDOM.ORG. *Analysis and Testing of RANDOM.ORG*. Disponible en: <https://www.random.org/analysis/>

- [4] Python Software Foundation. *random* — *Generate pseudo-random numbers*. Disponible en: <https://docs.python.org/3/library/random.html>
- [5] Matplotlib Development Team. *Matplotlib Documentation*. Disponible en: <https://matplotlib.org/stable/index.html>
- [6] Press, W. H., Teukolsky, S. A., Vetterling, W. T. y Flannery, B. P. *Numerical Recipes in C: The Art of Scientific Computing*, 2.^a ed. Cambridge University Press, 1992.
- [7] Cátedra de Simulación. *Material teórico de la materia*. Universidad Tecnológica Nacional — FRRO, 2026.
- [8] Cloudflare. *¿Por qué Cloudflare utiliza lámparas de lava para ayudar con la encriptación?* Disponible en: <https://www.cloudflare.com/es-es/learning/ssl/lava-lamp-encryption/>
- [9] Vayadan, D. *An Introduction to the Mersenne Twister Algorithm*. Medium. Disponible en: <https://medium.com/@vayadanderightna/an-introduction-to-the-mersenne-twister-algorithm-39f73dcabfed>